LAB 5: STATIC CODE ANALYSIS

Attendance Tracker - Before-Fix and After-Fix Report

SRN:PES2UG24CS922

Github Link: https://github.com/vikaskulakarni/SE_LAB_PES2UG24CS922

Part I — Before-Fix Analysis

Original program execution

```
(venv) vikas@LAPTOP-L5SQF22A:~/L5_61_attendance-tracker$ python attendance_tracker.py
R01 attendance %: 66
R02 attendance %: 50
Is 75% eligible? False
Shortage list: ['R01', 'R02']
R02 long absence? True
R02 latest date: 2026-9-3
Register entries for 2026-09-01: ['R01', 'R02', 'R02']
Exported to: attendance.csv
Reminder job: {'command': 'send_sms --shortage', 'shell': True}
ERP sync skipped (no network)
(venv) vikas@LAPTOP-L5SQF22A:~/L5_61_attendance-tracker$
```

Explanation: This screenshot establishes the runtime baseline before any fixes. It shows the original Attendance Tracker being executed, so the reader can compare the initial behavior with the corrected program in Part II. A successful run alone does not mean the code is safe or free of defects; that is why the static-analysis tools are also used.

Pylint baseline

```
(venv) vikas@LAPTOP-L5SQF22A:~/L5_61_attendance-tracker$ cat pylint_report.txt
************* Module attendance_tracker
attendance_tracker.py:60:0: C0301: Line too long (101/100) (line-too-long)
attendance_tracker.py:1:0: C0114: Missing module docstring (missing-module-docstring)
attendance_tracker.py:15:0: C0116: Missing function or method docstring (missing-function-docstring)
attendance_tracker.py:15:0: C0103: Function name "addStudent" doesn't conform to snake_case naming style (invalid-name)
attendance_tracker.py:18:0: C0116: Missing function or method docstring (missing-function-docstring)
attendance_tracker.py:18:0: C0103: Function name "markAttendance" doesn't conform to snake_case naming style (invalid-name)
attendance_tracker.py:18:0: W0102: Dangerous default value {} as argument (dangerous-default-value)
attendance_tracker.py:23:0: C0116: Missing function or method docstring (missing-function-docstring)
attendance_tracker.py:23:0: C0103: Function name "attendancePercentage" doesn't conform to snake_case naming style (invalid-name)
attendance_tracker.py:29:0: C0116: Missing function or method docstring (missing-function-docstring)
attendance_tracker.py:29:0: C0103: Function name "isEligible" doesn't conform to snake_case naming style (invalid-name)
attendance_tracker.py:30:11: R1719: The if expression can be replaced with 'test' (simplifiable-if-expression)
attendance_tracker.py:32:0: C0116: Missing function or method docstring (missing-function-docstring)
attendance_tracker.py:32:0: C0103: Function name "shortageList" doesn't conform to snake_case naming style (invalid-name)
attendance_tracker.py:35:11: C0121: Comparison 'isEligible(attendancePercentage(roll_no)) == False' should be 'isEligible(attendancePercentage(roll_no)) is F
alse' if checking for the singleton value False, or 'not isEligible(attendancePercentage(roll_no))' if testing for falsiness (singleton-comparison)
attendance_tracker.py:41:0: C0116: Missing function or method docstring (missing-function-docstring)
attendance_tracker.py:41:0: C0103: Function name "hasLongAbsence" doesn't conform to snake_case naming style (invalid-name)
attendance_tracker.py:43:11: R1729: Use a generator instead 'any(statuses[i:i + days] == ['A'] * days for i in range(len(statuses)))' (use-a-generator)
attendance_tracker.py:45:0: C0116: Missing function or method docstring (missing-function-docstring)
attendance_tracker.py:45:0: C0103: Function name "latestDate" doesn't conform to snake_case naming style (invalid-name)
attendance_tracker.py:46:12: R1721: Unnecessary use of a comprehension, use list([r[0] for r in students[roll_no]['records']]) instead. (unnecessary-comprehe
nsion)
attendance_tracker.py:49:0: C0116: Missing function or method docstring (missing-function-docstring)
attendance_tracker.py:49:0: C0103: Function name "exportCsv" doesn't conform to snake_case naming style (invalid-name)
attendance_tracker.py:50:8: W1514: Using open without explicitly specifying an encoding (unspecified-encoding)
attendance_tracker.py:53:4: C0206: Consider iterating with .items() (consider-using-dict-items)
attendance_tracker.py:50:8: R1732: Consider using 'with' for resource-allocating operations (consider-using-with)
attendance_tracker.py:58:0: C0116: Missing function or method docstring (missing-function-docstring)
attendance_tracker.py:58:0: C0103: Function name "syncToErp" doesn't conform to snake_case naming style (invalid-name)
attendance_tracker.py:60:15: W3101: Missing timeout argument for method 'requests.post' can cause your program to hang indefinitely (missing-timeout)
attendance_tracker.py:63:0: C0116: Missing function or method docstring (missing-function-docstring)
attendance_tracker.py:63:0: C0103: Function name "backupReports" doesn't conform to snake_case naming style (invalid-name)
attendance_tracker.py:64:4: R1732: Consider using 'with' for resource-allocating operations (consider-using-with)
attendance_tracker.py:66:0: C0116: Missing function or method docstring (missing-function-docstring)
attendance_tracker.py:66:0: C0103: Function name "legacyChannel" doesn't conform to snake_case naming style (invalid-name)
attendance_tracker.py:69:0: C0116: Missing function or method docstring (missing-function-docstring)
attendance_tracker.py:69:0: C0103: Function name "runJob" doesn't conform to snake_case naming style (invalid-name)
attendance_tracker.py:72:0: C0116: Missing function or method docstring (missing-function-docstring)
```

```
attendance_tracker.py:72:0: C0116: Missing function or method docstring (missing-function-docstring)
attendance_tracker.py:72:0: C0103: Function name "scheduleReminders" doesn't conform to snake_case naming style (invalid-name)
attendance_tracker.py:75:0: C0116: Missing function or method docstring (missing-function-docstring)
attendance_tracker.py:1:0: W0611: Unused import os (unused-import)
attendance_tracker.py:4:0: W0611: Unused import json (unused-import)
attendance_tracker.py:6:0: W0611: Unused datetime imported from datetime (unused-import)

-----------------------------------
Your code has been rated at 4.55/10
```

Explanation: Pylint reported a baseline score of 4.55/10. This low score indicates that the original source had multiple code-quality and maintainability issues. The score provides a measurable starting point for checking whether the revised source follows Pylint's rules more closely.

Bandit baseline

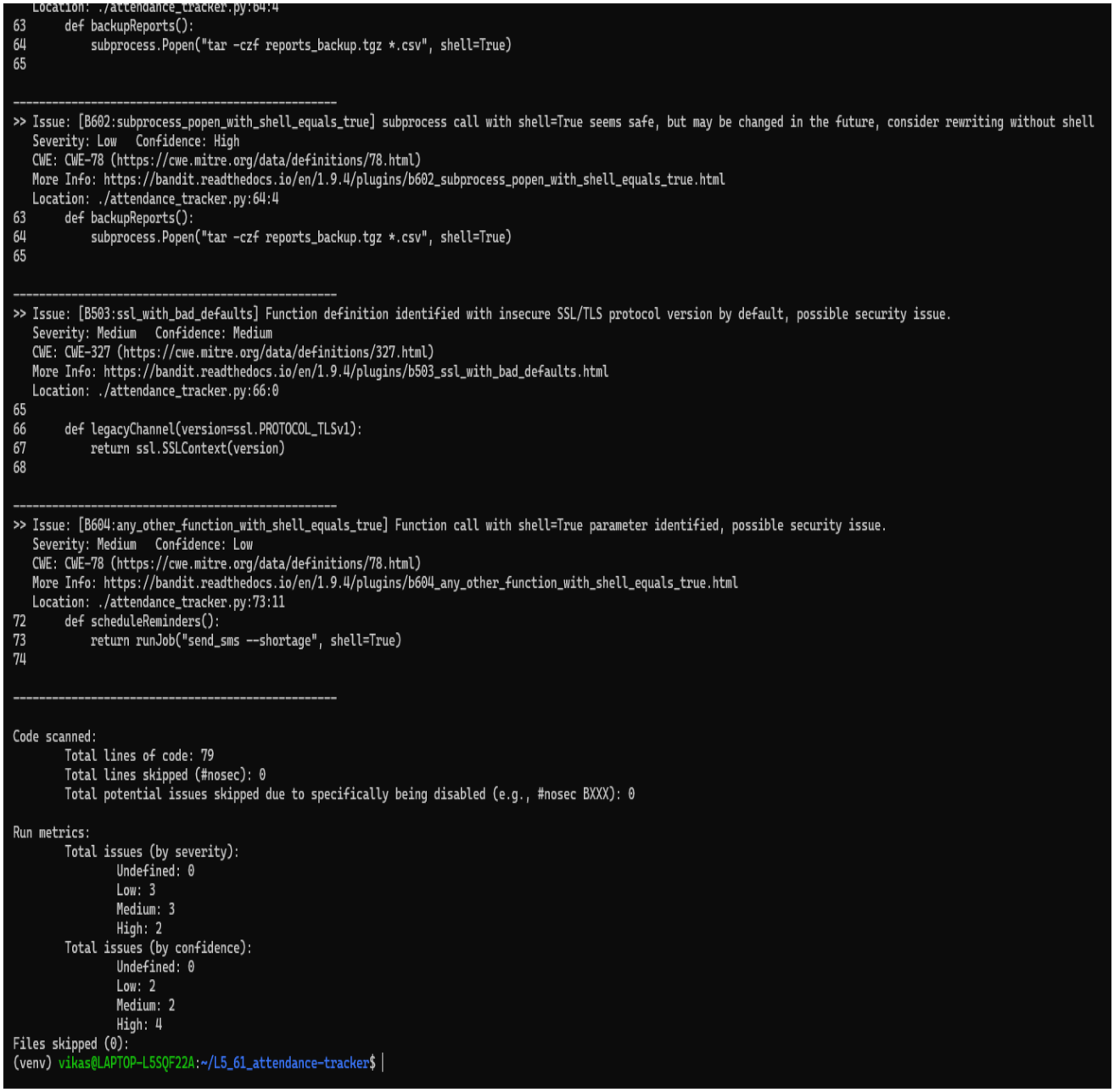
```
(venv) vikas@LAPTOP-L5SQF22A:~/L5_61_attendance-tracker$ python -m bandit -r attendance_tracker.py > bandit_report.txt
[main]  INFO    profile include tests: None
[main]  INFO    profile exclude tests: None
[main]  INFO    cli include tests: None
[main]  INFO    cli exclude tests: None
[main]  INFO    running on Python 3.12.3
(venv) vikas@LAPTOP-L5SQF22A:~/L5_61_attendance-tracker$ cat bandit_report.txt
Run started:2026-10-14 06:35:32.629835+00:00

Test results:
>> Issue: [B404:blacklist] Consider possible security implications associated with the subprocess module.
   Severity: Low   Confidence: High
   CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_imports.html#b404-import-subprocess
   Location: ./attendance_tracker.py:5:0
4       import json
5       import subprocess
6       from datetime import datetime

--------------------------------------------------
>> Issue: [B501:request_with_no_cert_validation] Call to requests with verify=False disabling SSL certificate checks, security issue.
   Severity: High   Confidence: High
   CWE: CWE-295 (https://cwe.mitre.org/data/definitions/295.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b501_request_with_no_cert_validation.html
   Location: ./attendance_tracker.py:60:15
59          payload = {"roll_no": roll_no, "pct": attendancePercentage(roll_no)}
60          response = requests.post(ERP_URL, json=payload, headers={"X-API-Key": ERP_API_KEY}, verify=False)
61          return response.status_code

--------------------------------------------------
>> Issue: [B113:request_without_timeout] Call to requests without timeout
   Severity: Medium   Confidence: Low
   CWE: CWE-400 (https://cwe.mitre.org/data/definitions/400.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b113_request_without_timeout.html
   Location: ./attendance_tracker.py:60:15
59          payload = {"roll_no": roll_no, "pct": attendancePercentage(roll_no)}
60          response = requests.post(ERP_URL, json=payload, headers={"X-API-Key": ERP_API_KEY}, verify=False)
61          return response.status_code

--------------------------------------------------
>> Issue: [B609:linux_commands_wildcard_injection] Possible wildcard injection in call: subprocess.Popen
   Severity: High   Confidence: Medium
   CWE: CWE-155 (https://cwe.mitre.org/data/definitions/155.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b609_linux_commands_wildcard_injection.html
   Location: ./attendance_tracker.py:64:4
63      def backupReports():
64          subprocess.Popen("tar -czf reports_backup.tgz *.csv", shell=True)
```



Explanation: This first section of the Bandit output shows security-related patterns detected in the original code. Bandit findings are prompts for review: each one should be checked against the code and its context, then corrected where appropriate. The findings helped identify risky practices rather than merely formatting problems and across the full baseline scan, eight findings were reported: two high, three medium, and three low. This severity breakdown helps prioritize security work, with higher-severity findings reviewed first.

Flake8 baseline

```
(venv) vikas@LAPTOP-L5SQF22A:~/L5_61_attendance-tracker$ python -m flake8 attendance_tracker.py > flake8_report.txt
(venv) vikas@LAPTOP-L5SQF22A:~/L5_61_attendance-tracker$ cat flake8_report.txt
attendance_tracker.py:1:1: F401 'os' imported but unused
attendance_tracker.py:4:1: F401 'json' imported but unused
attendance_tracker.py:6:1: F401 'datetime.datetime' imported but unused
attendance_tracker.py:15:1: E302 expected 2 blank lines, found 1
attendance_tracker.py:18:1: E302 expected 2 blank lines, found 1
attendance_tracker.py:23:1: E302 expected 2 blank lines, found 1
attendance_tracker.py:29:1: E302 expected 2 blank lines, found 1
attendance_tracker.py:32:1: E302 expected 2 blank lines, found 1
attendance_tracker.py:35:54: E712 comparison to False should be 'if cond is False:' or 'if not cond:'
attendance_tracker.py:41:1: E302 expected 2 blank lines, found 1
attendance_tracker.py:43:80: E501 line too long (84 > 79 characters)
attendance_tracker.py:45:1: E302 expected 2 blank lines, found 1
attendance_tracker.py:49:1: E302 expected 2 blank lines, found 1
attendance_tracker.py:54:80: E501 line too long (92 > 79 characters)
attendance_tracker.py:58:1: E302 expected 2 blank lines, found 1
attendance_tracker.py:60:80: E501 line too long (101 > 79 characters)
attendance_tracker.py:63:1: E302 expected 2 blank lines, found 1
attendance_tracker.py:66:1: E302 expected 2 blank lines, found 1
attendance_tracker.py:69:1: E302 expected 2 blank lines, found 1
attendance_tracker.py:72:1: E302 expected 2 blank lines, found 1
attendance_tracker.py:75:1: E302 expected 2 blank lines, found 1
attendance_tracker.py:78:80: E501 line too long (87 > 79 characters)
attendance_tracker.py:81:80: E501 line too long (87 > 79 characters)
attendance_tracker.py:82:80: E501 line too long (86 > 79 characters)
attendance_tracker.py:96:1: E305 expected 2 blank lines after class or function definition, found 1
(venv) vikas@LAPTOP-L5SQF22A:~/L5_61_attendance-tracker$
```

Explanation: Flake8 identified style and common-code issues such as unused imports, incorrect spacing, long lines, and Boolean comparisons. These issues can make code harder to read and maintain, and unused imports can also indicate unnecessary dependencies or leftover code. The revised file should be rescanned to confirm that these violations are gone.

Initial issue log

```
# Lab 5: Static Code Analysis – Issue Log

| Issue | Type | Line(s) | Description | Fix Approach |
|---|---|---|---|---|
| Mutable default argument | Bug | 18 | `register={}` is shared between function calls | Use `None` and initialize a new dictionary inside the function |
| SSL certificate verification disabled | Security | 60 | `verify=False` disables HTTPS certificate checks | Enable certificate verification |
| Request without timeout | Security / Reliability | 60 | The HTTP request can wait indefinitely | Add a suitable timeout |
| Unsafe subprocess wildcard handling | Security | 64 | Shell command uses a wildcard with `shell=True` | Replace shell execution with safer argument handling |
| Legacy TLS protocol | Security | 66 | Uses `ssl.PROTOCOL_TLSv1` | Use a modern secure TLS configuration |
| Unused imports | Style | 1, 4, 6 | Imports are not used | Remove unused imports |
| Incorrect comparison to False | Style | 35 | Uses `== False` | Use `not` |
| Long lines and spacing | Style | Multiple | Lines exceed the configured limit and blank lines are missing | Reformat according to PEP 8 |
```

Explanation: The issue log turns tool output into an actionable fix list. It records the problem, why it matters, and the planned correction, making the work traceable from the initial finding to the code change and final verification.

Initial issue summary

Tool	Before-fix result	Meaning
Pylint	4.55/10	Code-quality findings required review.
Bandit	8 findings: 2 high, 3 medium, 3 low	Potential security risks required review.
Flake8	Multiple violations	Formatting and selected static-check issues required correction.

Issue analysis and planned handling

Issue / pattern	Why it matters	Planned handling	Verification
Unused imports and style violations	Unused imports, inconsistent spacing, long lines, and Boolean comparisons reduce clarity and consistency.	Remove unnecessary imports, reformat lines, apply correct spacing, and simplify Boolean logic.	Pylint score 10.00/10; Flake8 exit code 0.
Mutable default register	A dictionary default can be shared between calls and create unexpected state.	Use None as the default and initialize a dictionary when needed; pass the shared register explicitly in the demo.	Program completed and printed register entries.
TLS verification disabled	Disabling certificate verification weakens HTTPS server authentication.	Use the default certificate verification and add a request timeout.	Bandit reported no issues; live ERP request was not tested.
Hardcoded API key	Secrets embedded in source code can be exposed through repository history or source sharing.	Read ERP_API_KEY from an environment variable.	Source reviewed; live ERP integration was not tested.
Shell command for backups	Shell execution can create security risks, and wildcard expansion changes when shell use is removed.	Use Python tarfile and explicitly select CSV files.	Bandit reported no issues; archive contents should be tested.
Legacy TLS protocol	The legacy protocol constant does not use modern platform TLS defaults.	Use ssl.create_default_context().	Bandit reported no issues.
Empty attendance records	Dividing by zero causes an exception when a student has no records.	Return 0 when the record list is empty.	Code updated; targeted empty-record test recommended.
Eligibility boundary	Using > makes a student with exactly 75% attendance ineligible.	Use >= for the 75% minimum threshold.	Demo output confirms 75% is eligible.
Date comparison and validation	String comparisons can misorder inconsistent date formats.	Parse dates using the expected YYYY-MM-DD format before comparing.	Demo displays the latest date; malformed-date test recommended.
CSV formula-like values	Spreadsheet applications may interpret formula-like text as formulas.	Prefix values beginning with =, +, -, or @ before exporting.	CSV export succeeded; inspect exported contents to verify.

Part II — After-Fix Analysis

Successful program execution

```
(venv) vikas@LAPTOP-L5SQF22A:~/L5_61_attendance-tracker$ python attendance_tracker.py
R01 attendance %: 66
R02 attendance %: 50
Is 75% eligible? True
Shortage list: ['R01', 'R02']
R02 long absence? True
R02 latest date: 2026-09-04
Register entries for 2026-09-01: ['R01', 'R02', 'R02']
Exported to: attendance.csv
Reminder job: {'command': ['send_sms', '--shortage'], 'shell': False}
ERP sync skipped (no network)
(venv) vikas@LAPTOP-L5SQF22A:~/L5_61_attendance-tracker$ python -m pylint attendance_tracker.py > /tmp/pylint_after.txt
python -m bandit -r attendance_tracker.py > /tmp/bandit_after.txt
python -m flake8 attendance_tracker.py > /tmp/flake8_after.txt
[main]  INFO    profile include tests: None
[main]  INFO    profile exclude tests: None
[main]  INFO    cli include tests: None
[main]  INFO    cli exclude tests: None
[main]  INFO    running on Python 3.12.3
(venv) vikas@LAPTOP-L5SQF22A:~/L5_61_attendance-tracker$ tail -n 5 /tmp/pylint_after.txt

-------------------------------------------------------------------
Your code has been rated at 10.00/10 (previous run: 4.55/10, +5.45)
```

Explanation: The corrected program runs to completion without a traceback and displays attendance percentages, eligibility, shortage information, date information, CSV export, and the reminder-job configuration. This is useful runtime evidence, but it does not replace targeted tests for edge cases or external integrations.

Pylint after-fix

```
(venv) vikas@LAPTOP-L5SQF22A:~/L5_61_attendance-tracker$ tail -n 5 /tmp/pylint_after.txt

-------------------------------------------------------------------
Your code has been rated at 10.00/10 (previous run: 4.55/10, +5.45)
```

Explanation: Pylint reports 10.00/10 after the changes, compared with 4.55/10 before the changes. This confirms that the final source satisfies the checks applied by this Pylint run. It should be interpreted as a code-quality score, not as proof that the application has no bugs.

Bandit after-fix

```
(venv) vikas@LAPTOP-L5SQF22A:~/L5_61_attendance-tracker$ cat /tmp/bandit_after.txt
Run started:2026-10-14 08:25:50.982800+00:00

Test results:
        No issues identified.

Code scanned:
        Total lines of code: 159
        Total lines skipped (#nosec): 0
        Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
        Total issues (by severity):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 0
        Total issues (by confidence):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 0
Files skipped (0):
```

Explanation: Bandit reports no issues identified in the scanned source after the corrections. This indicates that the patterns detected in the baseline are no longer being reported by this scan. It does not guarantee that every possible security vulnerability has been eliminated.

Flake8 after-fix

```
(venv) vikas@LAPTOP-L5SQF22A:~/L5_61_attendance-tracker$ python -m flake8 attendance_tracker.py
echo "Flake8 exit code: $?"
Flake8 exit code: 0
```

Explanation: Flake8 produced no violation output and returned exit code 0. Together, these indicate that the file passed the configured Flake8 checks. Keeping the exit code alongside an empty report makes the result easier to verify reproducibly.

Saved after-fix reports

```
(venv) vikas@LAPTOP-L5SQF22A:~/L5_61_attendance-tracker$ mkdir -p evidence/after/screenshots

python -m pylint attendance_tracker.py > evidence/after/pylint_report.txt
python -m bandit -r attendance_tracker.py > evidence/after/bandit_report.txt 2>&1
python -m flake8 attendance_tracker.py > evidence/after/flake8_report.txt

ls -l evidence/after/
total 12
-rw-r--r-- 1 vikas vikas  621 Oct 14 08:32 bandit_report.txt
-rw-r--r-- 1 vikas vikas    0 Oct 14 08:32 flake8_report.txt
-rw-r--r-- 1 vikas vikas  140 Oct 14 08:32 pylint_report.txt
drwxr-xr-x 2 vikas vikas 4096 Oct 14 07:52 screenshots
(venv) vikas@LAPTOP-L5SQF22A:~/L5_61_attendance-tracker$
```

Explanation: This screenshot shows the saved after-fix reports for Pylint, Bandit, and Flake8 in the evidence/after/ directory. Saving tool output makes the result auditable and allows it to be compared with the before-fix reports. The before and after evidence should both be retained.

Final results

Tool	Before-fix	After-fix
Pylint	4.55/10	10.00/10
Bandit	8 findings	No issues identified
Flake8	Multiple violations	No violations; exit code 0

Corrections recorded

- Replaced the mutable default argument and passed a shared register explicitly in the demo.
- Removed the hardcoded ERP API key; restored HTTPS certificate verification and added a request timeout.
- Replaced shell-based report archiving and legacy TLS configuration with safer alternatives.
- Handled empty attendance records, clarified the 75% eligibility boundary, and parsed dates before comparison.
- Added CSV formula-like value protection and used a context manager for CSV export.
- Improved naming, Boolean logic, and formatting.

Verification note: the demo completed without a traceback; Pylint reported 10.00/10, Bandit reported no issues, and Flake8 returned exit code 0. The ERP request was not sent to a live service. Targeted functional checks for empty records, invalid dates, CSV protection, and backup contents remain recommended.

Reflection and Good Practice

1. What is static code analysis, and why is it useful?

Static code analysis examines source code without running the application. It helps identify likely defects, insecure coding patterns, and style problems early, making code easier to review and maintain.

2. What did the three tools contribute?

Pylint highlighted code-quality and maintainability issues, Bandit checked for common security risks, and Flake8 identified style and formatting violations. Using the tools together provided broader coverage than relying on a single tool.

3. What changed after fixing the findings?

The reported Pylint score improved from 4.55/10 to 10.00/10, Bandit went from eight findings to no identified issues, and Flake8 returned no violations. These results show that the reported static-analysis findings were addressed in the revised source.

4. What are the limitations of static analysis?

A clean static-analysis report does not prove that a program is bug-free or secure. Tools can miss runtime, integration, and business-logic problems. Functional tests and code review are still needed, especially for edge cases and external-service behavior.

Verification Summary

The recorded after-fix evidence shows the following results:

Check	Recorded result	Interpretation
Program execution	Completed without a traceback in the recorded demo	Basic execution path succeeded.
Pylint	10.00/10	Above the required threshold of 9.5.
Bandit	No issues identified	No findings were reported for the scanned source.
Flake8	Exit code 0; no output	No violations were reported.

Reproducibility: rerun the tools from the repository root and save their output separately for the before and after states. Keep the original source and before-fix reports unchanged so the comparison remains traceable.